PRELIMINARY EXAMINATIONS 2026

Course 1: Introduction to Computer Science

A Comprehensive Academic Guide to Computer Architectures, Operating System Mechanics, Software Engineering, and Modern Computing Technologies

COURSE CODE
CS-101

ACADEMIC YEAR
2026

PUBLISHER
Preliminary Examinations Press

TARGET AUDIENCE
Undergraduate & Preparatory Students

Table of Contents

MODULE 1

Foundations of Computing

1.1 Early Mechanical Computing to Modern Systems

The history of computation dates back thousands of years. Early humans used visual counters like the **Abacus** for basic addition and subtraction. In 1642, Blaise Pascal invented the **Pascaline**, a gear-driven calculator that could add and subtract directly. Later, in 1837, mathematician Charles Babbage designed the **Analytical Engine**. This conceptual design featured an Arithmetic Logic Unit (which he called the "Mill") and memory storage (which he called the "Store"), forming the blueprint for modern processors. Ada Lovelace wrote notes on the Analytical Engine, documenting the first algorithm to calculate Bernoulli numbers, earning her recognition as the world's first computer programmer.

The development of electronic computing in the 20th century was driven by the need for faster calculations during wartime and industrial expansion. The evolution of hardware is divided into five distinct generations, each defined by its underlying electronic component:

- **First Generation (1940-1956):** Powered by *vacuum tubes*. These machines were massive, filled entire rooms, and consumed enormous amounts of electricity. They broke down frequently and were programmed using low-level machine code (binary). Famous examples include the ENIAC and UNIVAC.
- **Second Generation (1956-1963):** Introduced the *transistor*, which replaced vacuum tubes. Transistors were smaller, faster, cheaper, and more energy-efficient. This generation saw the development of assembly languages and early high-level programming languages like FORTRAN and COBOL.
- **Third Generation (1964-1971):** Enabled by the *Integrated Circuit (IC)*, which placed hundreds of transistors onto a single silicon chip. This development led to smaller, more affordable systems, the introduction of keyboards and monitors, and early operating systems that allowed multiple programs to run simultaneously.
- **Fourth Generation (1971-Present):** Defined by the *microprocessor*, created through Large Scale Integration (LSI) and Very Large Scale Integration (VLSI), which combined millions of transistors onto a single microchip. This technology powered the personal computer (PC) revolution and led to the creation of the Internet, laptops, and mobile devices.

- **Fifth Generation (Future & Beyond):** Focuses on *Artificial Intelligence (AI)*, utilizing ultra-large-scale integration (ULSI), parallel processing, and quantum computing. The goal is to build devices that can process natural language, adapt to new inputs, and perform parallel computations.

Figure 1.1: Historical timeline of computing generations from vacuum tubes to modern artificial intelligence.

1.2 Characteristics, Applications & Types of Computers

Modern computers possess key performance characteristics that make them essential for high-throughput calculation and analysis:

- **Speed:** CPU speeds are measured in gigahertz (GHz), meaning billions of clock cycles per second. A CPU can execute an instruction in fractions of a nanosecond. Higher-level system speeds are calculated in Millions of Instructions Per Second (MIPS) or Floating-Point Operations Per Second (FLOPS) for scientific supercomputers.
- **Accuracy & GIGO:** Computers are highly precise, calculating to arbitrary floating-point limits. Errors are almost always the result of incorrect human code or input data. This concept is known as **GIGO (Garbage In, Garbage Out)**.
- **Diligence:** Unlike humans, processors do not experience fatigue or loss of concentration, running millions of consecutive tasks with consistent accuracy.
- **Versatility:** Computers can perform a wide range of tasks, from processing text files to executing complex scientific models.
- **Storage Capacity:** Modern systems can store large amounts of data in microscopic physical spaces, ranging from Megabytes to Petabytes.

Classification of Modern Computers

Computers are grouped into classes depending on their physical scale and calculation throughput:

1. **Supercomputers:** Highly parallel clusters of processors designed for intensive simulations (e.g., climate modeling, nuclear simulation). These systems process data in

Petaflops and require specialized cooling.

2. **Mainframe Computers:** Large, reliable central servers designed for high-volume transaction processing and input/output bandwidth. Used primarily by banking, insurance, and government organizations.
3. **Minicomputers (Midrange):** Multi-user computers that are smaller than mainframes. Historically used by medium-sized businesses, they have largely been replaced by clusters of standard servers.
4. **Microcomputers:** Everyday computing systems powered by a single CPU chip (e.g., desktops, laptops, tablets, smartphones, and embedded microcontrollers).

1.3 Data vs Information Pipeline & Logic Gates

In computer science, a key concept is the distinction between data and information:

- **Data:** Unstructured, raw symbols, numbers, or characters lacking context (e.g., a list of numbers: `[22.4, 25.8, 19.1]`).
- **Information:** Data that has been processed, structured, and presented in a meaningful context (e.g., `"The average room temperatures in Celsius recorded on Monday are 22.4, 25.8, and 19.1."`).

Introduction to Logic Gates

At the lowest level, computers process raw binary data using electrical circuits called **Logic Gates**. These gates implement boolean logic. The three primary gates are:

- **AND Gate:** Outputs 1 only if all inputs are 1. Represented mathematically as $Y = A \cdot B$.
- **OR Gate:** Outputs 1 if at least one input is 1. Represented mathematically as $Y = A + B$.
- **NOT Gate (Inverter):** Inverts the input (outputs 1 if input is 0, and 0 if input is 1). Represented mathematically as $Y = \bar{A}$.

Other gates include **NAND** (NOT AND), **NOR** (NOT OR), and **XOR** (Exclusive OR), which outputs 1 if inputs are different. NAND and NOR gates are called **universal gates** because any boolean function can be implemented using only NAND or only NOR gates.

Input A	Input B	A AND B	A OR B	A XOR B	NOT A
0	0	0	0	0	1
0	1	0	1	1	1
1	0	0	1	1	0
1	1	1	1	0	0

1.4 Number Systems: Base Conversions

All digital hardware represents data as electrical voltages (low/high, corresponding to binary 0 and 1). To understand how data is represented, we study different mathematical bases:

Decimal (Base 10) to Binary (Base 2) Conversion

Convert decimal numbers to binary by dividing successively by 2 and reading the remainders bottom-up (least significant bit to most significant bit).

```
Example: Convert Decimal 57 to Binary.
57 / 2 = 28 remainder 1 (Least Significant Bit)
28 / 2 = 14 remainder 0
14 / 2 = 7 remainder 0
7 / 2 = 3 remainder 1
3 / 2 = 1 remainder 1
1 / 2 = 0 remainder 1 (Most Significant Bit)
Result (read bottom-up): 111001₂
```

Hexadecimal (Base 16) to Decimal (Base 10) Conversion

To convert from Hexadecimal (digits 0-9, A-F where A=10, F=15) to Decimal, multiply each digit by its positional power of 16.

```
Example: Convert Hexadecimal 1F3 to Decimal.
1F3₁₆ = (1 × 16²) + (F × 16¹) + (3 × 16⁰)
Since F = 15:
= (1 × 256) + (15 × 16) + (3 × 1)
= 256 + 240 + 3
= 499₁₀
```

1.5 Computer Architecture vs Organization & Software Types

Computer Architecture refers to the logical design and attributes of a system visible to an assembly programmer (e.g., instruction set architecture, word widths, memory addressing modes). **Computer Organization** refers to the physical implementation of that architecture (e.g., circuit schematics, hardware buses, memory interface clocks, and cache sizes).

Software is divided into two main categories:

- **System Software:** Manages the hardware resources and provides services for other software (e.g., OS kernel, device drivers, system utilities, compilers, assemblers).
- **Application Software:** Performs user tasks (e.g., browsers, word processors, databases, video games).
- **Open Source vs Proprietary:** Open-source projects share their underlying source code freely (e.g., Linux, Apache, Python), allowing community modification. Proprietary software keeps it closed and licensed (e.g., Windows, macOS, Photoshop).

Module 1 - Practice Set & Exercises

1. Compare Vacuum Tubes and Transistors. What physical and electrical limitations of the first generation prompted the development of the second generation?
2. Classify the following systems as either supercomputers, mainframes, or microcomputers: (a) A system simulating aerodynamic flows on a space shuttle, (b) A banking system processing 10,000 credit card entries per second, (c) A modern thin laptop.
3. Explain the concept of Garbage In, Garbage Out (GIGO) using a database validation example.
4. Convert the decimal number 156 into its binary representation. Show all steps of division.
5. Convert the hexadecimal value $2C5_{16}$ to its decimal equivalent. Show your work for each positional weight.
6. Write out the truth table for a 2-input NAND gate. Why is the NAND gate referred to as a "universal gate" in computer engineering?
7. Differentiate between computer architecture and computer organization using the instruction set vs. ALU hardware implementation as examples.
8. Explain why Application Software cannot run directly on the physical CPU without System Software.
9. Discuss the security trade-offs of using open-source software compared to closed-source proprietary software.

MODULE 2

Computer Hardware & System Components

2.1 CPU Core Mechanics & Control Units

The Central Processing Unit (CPU) is the core execution engine of a computer. It operates on a continuous cycle known as the **Fetch-Decode-Execute Cycle**. In the *Fetch* stage, the CPU retrieves instruction bytes from memory. In the *Decode* stage, the Control Unit (CU) interprets the instruction to determine the required operation. In the *Execute* stage, the Arithmetic Logic Unit (ALU) performs the operation, and the result is written back to memory or registers.

The CPU architecture is traditionally based on the **Von Neumann Architecture**, which features a single memory space for both instructions and data connected to the processor via shared buses. A limitation of this design is the **Von Neumann Bottleneck**, where throughput is limited because the CPU cannot read instructions and read/write data at the same time. The alternative is the **Harvard Architecture**, which uses separate physical memory paths and buses for instructions and data.

2.2 Internal CPU Registers & Cache Coherency

The CPU uses specialized internal registers to hold data and states during execution:

- **Program Counter (PC):** Holds the memory address of the next instruction code to be fetched.
- **Instruction Register (IR):** Holds the instruction code that has just been fetched from memory while it is being decoded.
- **Memory Address Register (MAR):** Holds the memory address that the CPU needs to read from or write to.
- **Memory Data Register (MDR):** Holds the actual data fetched from memory or prepared to be written into memory.
- **Accumulator (AC):** Holds the output of the most recent ALU computation.

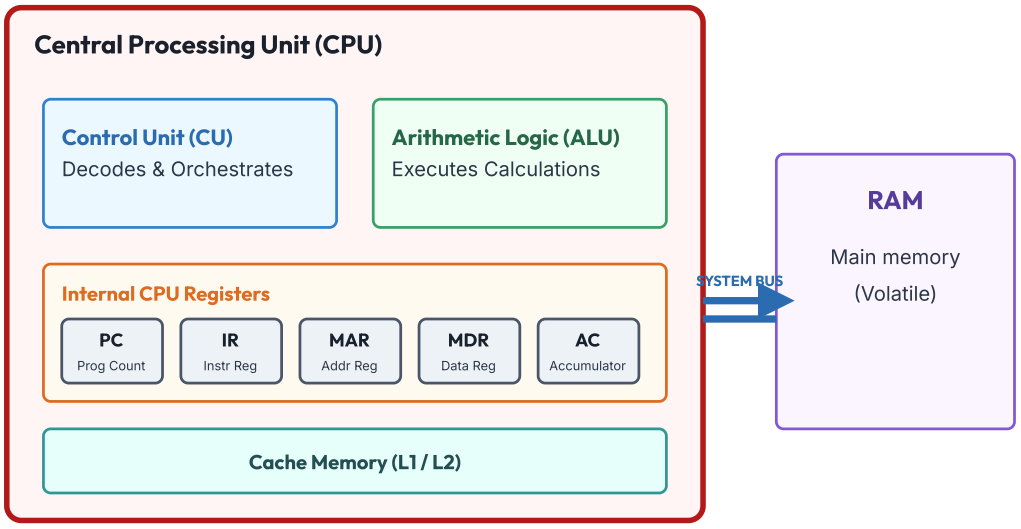

Figure 2.1: CPU registers and their physical connection to RAM.

2.3 Memory Hierarchy & Primary Memory

To optimize performance, computers use a hierarchical memory model. This model balances speed, capacity, and cost per byte:

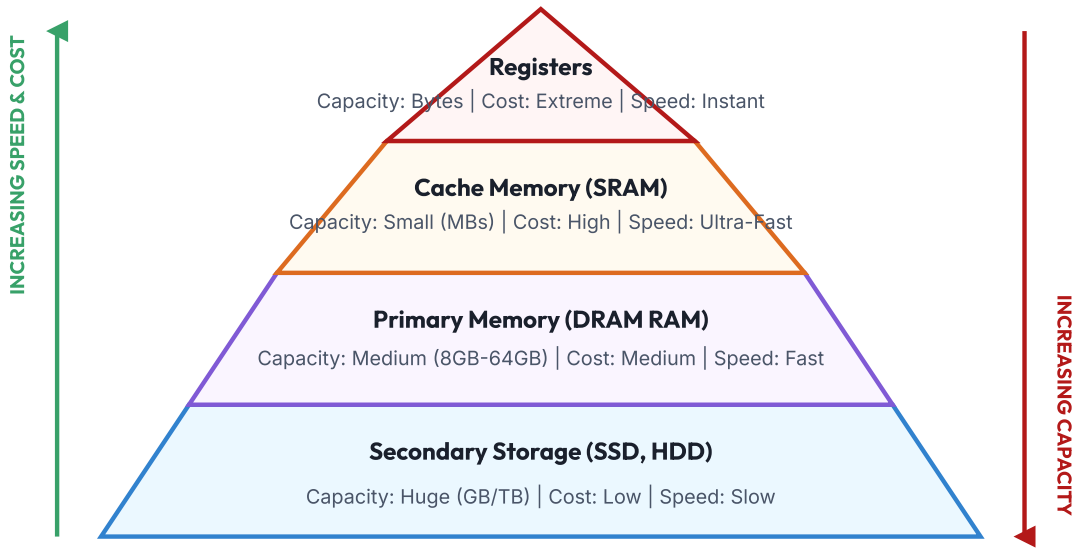

Figure 2.2: Memory hierarchy pyramid showing speed, capacity, and cost trends.

Primary Memory: RAM vs ROM

- **RAM (Random Access Memory):** Volatile, read-write memory containing files and program states in active execution. RAM is split into two architectures:
  - *DRAM (Dynamic RAM):* Uses a single transistor and capacitor pair per bit. Capable of high densities, but capacitors leak charge and must be refreshed millions of times per second.
  - *SRAM (Static RAM):* Uses a six-transistor latch circuit per bit. It does not require refreshing and is faster than DRAM, but is less dense and more expensive. SRAM is primarily used for CPU L1/L2/L3 cache.

- **ROM (Read Only Memory):** Non-volatile memory that stores boot firmware. Types include:
  - *EEPROM:* Electrically Erasable Programmable ROM, which can be erased and rewritten electrically, allowing modern UEFI/BIOS firmware updates.

2.4 Secondary Storage (HDD vs SSD Flash) & Buses

Secondary storage provides persistent, non-volatile data storage.

- **HDD (Hard Disk Drive):** Reads and writes data magnetically using spinning metal platters and movable read/write heads. Mechanical components introduce rotational latency and seek times, making HDDs susceptible to physical damage.
- **SSD (Solid State Drive):** Uses NAND flash memory chips. SSDs have no moving parts, which minimizes read/write latency. They are physically quiet, shock-resistant, and offer higher data transfer rates than HDDs.

System Buses: The motherboard routes data between the CPU, RAM, and storage using physical lines divided into three buses:

- **Data Bus:** Moves data bits between system components.
- **Address Bus:** Transmits memory address location references from the CPU to RAM.
- **Control Bus:** Carries control and timing signals (e.g., Read/Write commands, clock cycles).

Module 2 - Practice Set & Exercises

1. Describe the step-by-step register actions during the Fetch stage of the CPU cycle.
2. Differentiate between DRAM and SRAM in terms of transistor count, speed, cost, and physical refreshing requirements.
3. Explain the concept of memory hierarchy latency. Why can a CPU not use SRAM exclusively as its main memory?
4. Explain why SSDs are faster and more durable than mechanical HDDs. Include how the presence of moving parts affects HDD read/write latency.
5. A system bus has a 32-bit address bus. Calculate the maximum physical memory capacity this address bus can reference in bytes. (Hint: 2^{32} bytes).
6. Describe the functions of the Northbridge and Southbridge chipsets in a motherboard architecture.
7. Explain the difference between PCIe (Peripheral Component Interconnect Express) and USB buses in terms of data transfer speeds and typical device attachments.
8. How does a CPU cache increase instructions-per-cycle (IPC) execution performance? Explain L1, L2, and L3 cache designs.

MODULE 3

Operating Systems Fundamentals

3.1 Core Architecture & Functions of an OS

An Operating System (OS) is the primary system software that manages hardware resources and provides common services for application programs. Without an OS, application developers would need to write low-level code to communicate directly with hardware, manage memory blocks, write disk blocks, and coordinate input devices.

3.2 Process Control Blocks & Threads

The OS tracks running programs as processes. A **Program** is a passive collection of instructions stored on disk (e.g., an executable file). A **Process** is an active program loaded into memory.

The OS represents each process using a data structure called a **Process Control Block (PCB)**. The PCB contains:

- **Process Identifier (PID):** A unique integer identification code.
- **Process State:** Indicates the current execution state of the process (New, Ready, Running, Waiting, Terminated).
- **Program Counter (PC):** Holds the address of the next instruction to execute for this process.
- **CPU Registers:** Saved register states when the process is paused.
- **Memory Limits:** Information on page tables and memory boundaries.

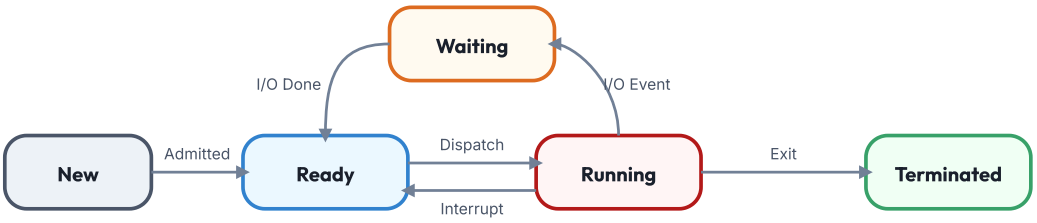

Figure 3.2: Five-state process model showing state transitions.

3.3 Process Scheduling & CPU Allocation Math

The OS scheduler manages CPU cores when multiple processes require execution. Scheduling algorithms balance performance and throughput:

- **FCFS (First-Come, First-Served):** A non-preemptive algorithm that processes requests in the order of arrival. This can cause smaller processes to wait behind longer ones, an issue known as the **Convoy Effect**.
- **Round Robin (RR):** A preemptive scheduling algorithm that assigns each process a fixed slice of execution time, known as a **Time Quantum**. If the quantum expires, the process is preempted and returned to the ready queue.
- **Shortest Job First (SJF):** Selects the process with the shortest execution time first, which minimizes average wait times. Can be preemptive or non-preemptive.

> **Scheduling Example: Average Waiting Time**
>
> Suppose three processes arrive in the order P1, P2, P3 at time 0. The execution time for each process is: P1 = 24ms, P2 = 3ms, P3 = 3ms.
> **Using FCFS:**
> * Waiting Time: P1 = 0ms, P2 = 24ms, P3 = 27ms.
> * Average Waiting Time: `(0 + 24 + 27) / 3 = 17ms`.
> **Using SJF:** (Executes P2, P3, then P1)
> * Waiting Time: P2 = 0ms, P3 = 3ms, P1 = 6ms.
> * Average Waiting Time: `(0 + 3 + 6) / 3 = 3ms`.

3.4 Memory Paging, Page Faults, and Swapping

Operating systems use **Virtual Memory** to execute processes larger than the available physical RAM. Memory is structured using:

- **Paging:** The OS divides virtual memory into fixed blocks called **Pages**, and divides physical RAM into blocks called **Frames**. Pages and frames are the same size (typically 4KB).
- **Page Table:** A table that maps virtual pages to physical frames, allowing virtual memory to be loaded into non-contiguous physical RAM.
- **Page Fault:** Occurs when the CPU references a page not currently loaded in physical RAM. The OS must retrieve the page from secondary storage, swapping it in.

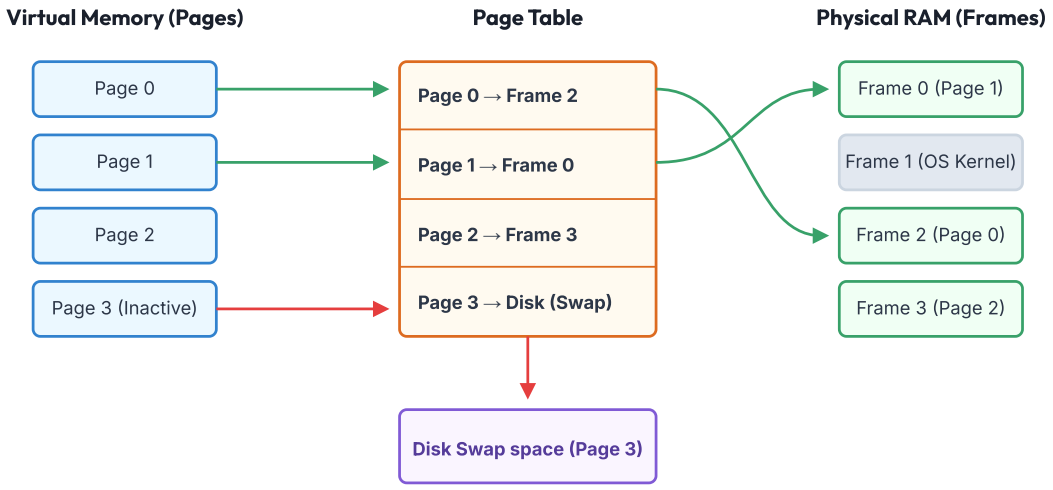

Figure 3.3: Virtual memory mapping process from pages to frames.

3.5 Bootloader, Kernel loading, and CLI Commands

The **Boot Sequence** launches the OS when the computer is powered on:

1. **POST (Power-On Self-Test):** Firmware in ROM initializes hardware components and checks for failures.
2. **BIOS/UEFI:** Resolves system settings and locates the bootable device sector.
3. **Bootloader:** A program (e.g., GRUB, Windows Boot Manager) loads the OS kernel into memory.
4. **OS Kernel:** The core OS starts system services and launches the user interface.

Core CLI Commands Reference

Operation	Windows Command	Linux / Unix Command	Description
List Files	`dir`	`ls`	Displays files and folders in the current directory.
Change Directory	`cd`	`cd`	Changes the active working directory.
Make Directory	`mkdir`	`mkdir`	Creates a new folder.
Copy File	`copy`	`cp`	Copies a file from source to destination.
Move File	`move`	`mv`	Moves or renames a file.
Delete File	`del`	`rm`	Permanently deletes a file.
View File	`type`	`cat`	Prints text file content directly to screen.
IP Details	`ipconfig`	`ifconfig`	Shows active network adapter details.

Module 3 - Practice Set & Exercises

1. What is the role of the operating system kernel? Differentiate between monolithic kernels and microkernels.
2. Explain the contents of a Process Control Block (PCB). Why must the PCB store register states?
3. Four processes, P1 (8ms), P2 (12ms), P3 (4ms), and P4 (6ms), arrive at time 0. Calculate the average waiting time using non-preemptive Shortest Job First (SJF) scheduling.
4. What is a page fault? Describe the steps the OS takes to handle a page fault.
5. Explain thrashing in virtual memory. Under what conditions does thrashing occur?
6. Compare NTFS, FAT32, and ext4 file systems based on maximum file size, compatibility, and journaling support.
7. Define the difference between cooperative multitasking and preemptive multitasking. Which model is used by modern operating systems?
8. Provide the terminal commands (Windows vs. Linux) to list all active files, copy a file named `textbook.txt`, and display network configuration details.

MODULE 4

Software Development & Networking

4.1 Algorithm Logical Structures & Flowchart Shapes

An **Algorithm** is a step-by-step logic procedure to solve a computational problem. It must be finite, unambiguous, and effective. Programmers design algorithms using:

- **Flowcharts:** Graphical shapes representing program execution flow. Ovals represent start/end points, parallelograms represent input/output operations, rectangles represent process steps, and diamonds represent decision branches.
- **Pseudocode:** A structured, human-readable draft of a program logic that mimics code patterns without rigid syntax rules.

Algorithm Design: Check Even or Odd

Pseudocode:

```
INPUT number
IF number MOD 2 EQUALS 0 THEN
    OUTPUT "Even"
ELSE
    OUTPUT "Odd"
ENDIF
```

4.2 SDLC Models: Waterfall vs Agile Iterations

The **Software Development Life Cycle (SDLC)** is a structured methodology for software creation:

SDLC Phase	Primary Objectives	Deliverables
Requirements Analysis	Gathering customer needs and defining system boundaries.	Software Requirements Specification (SRS)
System Design	Drafting database architecture, UML design, and UI wireframes.	Design Document Specification (DDS)
Implementation	Writing the source code in selected programming languages.	Source Code Repositories
Testing	Executing code to find and log software bugs.	Test Reports & Bug Logs
Deployment	Releasing the build to production servers.	Live Production Applications
Maintenance	Fixing bugs and releasing feature upgrades.	Patches & Performance Updates

Development teams typically follow one of two primary execution models:

1. **Waterfall Model:** A sequential approach where each phase must be completed before the next begins. This model is rigid but predictable, making it suitable for projects with stable requirements.
2. **Agile Model:** An iterative approach that focuses on continuous feedback, regular updates, and collaboration. Development is divided into short cycles called **Sprints**.

4.3 Compiler Stages vs Line-by-Line Interpretation

Programming languages are translated into binary machine code using compilers or interpreters:

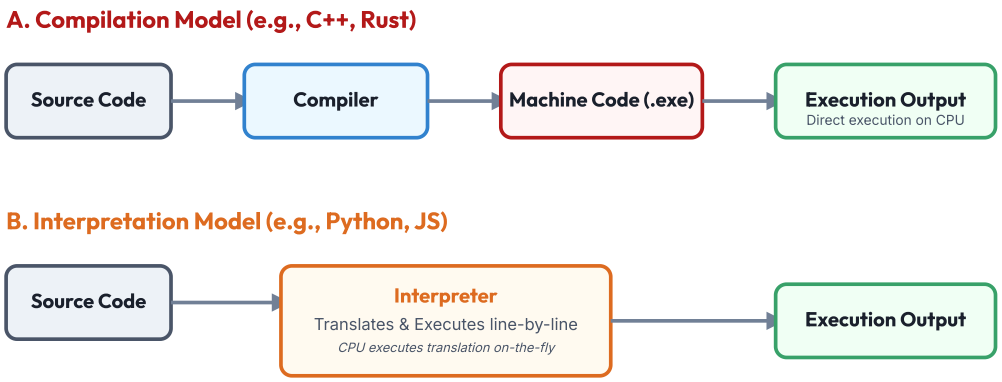

Figure 4.2: Compilers versus Interpreters processing pipelines.

4.4 Git Repositories, DNS Hierarchy, & HTTP/HTTPS Handshakes

Version Control (Git): Resolves the challenge of tracking changes when multiple developers work on the same codebase. `git commit` saves code snapshots to local history, while `git push` uploads changes to remote servers (e.g., GitHub).

DNS (Domain Name System): Translates domain names to IP addresses. When a user requests a domain name, the OS queries a root name server, which redirects the request to a Top-Level Domain (TLD) server (e.g., `.com`), and finally to an authoritative name server that returns the target IP address.

4.5 Client-Server Request Cycles & Cloud Paradigms

In the client-server model, the client browser initiates requests to a web server over the network:

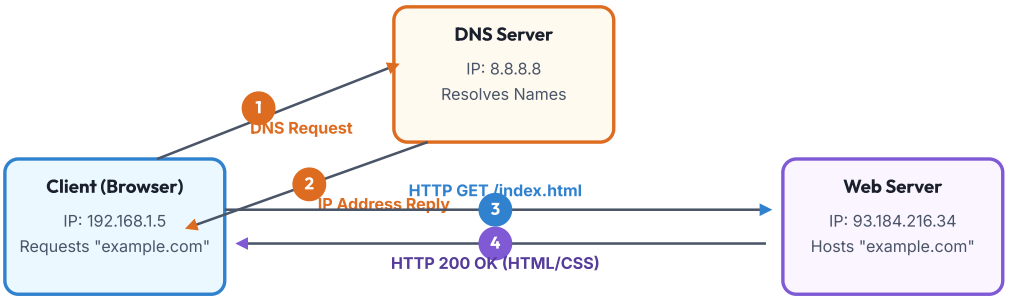

Figure 4.3: DNS resolution and HTTP request-response cycle.

Cloud Computing Models

- **IaaS (Infrastructure as a Service):** Provides raw compute power, storage, and networking resources (e.g., AWS EC2, Google Compute Engine).
- **PaaS (Platform as a Service):** Provides databases, runtimes, and developer tooling, abstracting operating system management (e.g., Heroku, AWS Elastic Beanstalk).
- **SaaS (Software as a Service):** Delivers fully functional applications over the web (e.g., Google Workspace, Microsoft 365).

Module 4 - Practice Set & Exercises

1. Draw a flowchart for an algorithm that reads 10 numbers from the user and outputs the average value.
2. Explain the key differences between the Waterfall and Agile software development models.
3. Describe the stages of compilation, from lexical analysis to code generation.
4. What is the difference between a Git merge conflict and a standard commit? How are merge conflicts resolved?
5. Explain the role of DNS. What happens if a DNS server fails to resolve a domain name?
6. How does HTTPS encrypt data? Differentiate between symmetric and asymmetric encryption in HTTPS.
7. What are the characteristics of cloud computing? Differentiate between IaaS, PaaS, and SaaS.
8. Define the three-way handshake protocol used by TCP to establish a network connection.

MODULE 5

Modern Computing & Career Paths

5.1 Relational Normalization vs NoSQL Document Engines

Databases store and manage application data. Relational databases (SQL) organize data into tables with predefined schemas. Non-relational databases (NoSQL) store unstructured data using documents, key-value pairs, or graphs.

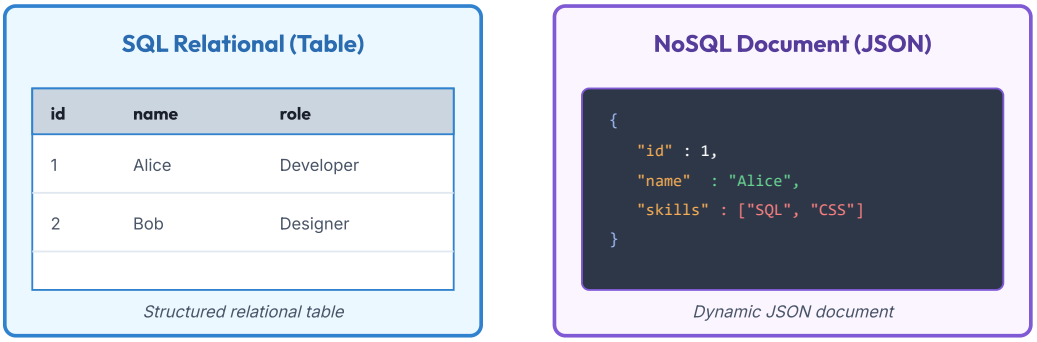

Figure 5.2: Comparison between SQL table structures and NoSQL JSON documents.

5.2 Artificial Intelligence, Machine Learning, and Data Science

Modern computing systems utilize data analysis to recognize patterns:

- **Artificial Intelligence (AI):** The broad field of creating software capable of simulating human reasoning, decision-making, and visual interpretation.
- **Machine Learning (ML):** A subset of AI where systems learn from data patterns without explicit programming.
  - *Supervised Learning:* Algorithms learn from labeled training data (e.g., classifying emails as spam or not spam).
  - *Unsupervised Learning:* Algorithms group unlabeled data based on patterns (e.g., segmenting customers by purchasing habits).
- **Data Science:** Cleaning, analyzing, and visualizing data to extract business insights.

5.3 Cryptography keys, Hashing, & Blockchain Ledgers

Cryptography secures data across public networks.

- **Symmetric Cryptography:** Uses a single shared key for encryption and decryption. Requires secure key sharing.
- **Asymmetric Cryptography:** Uses a paired key set: a public key for encryption and a private key for decryption.

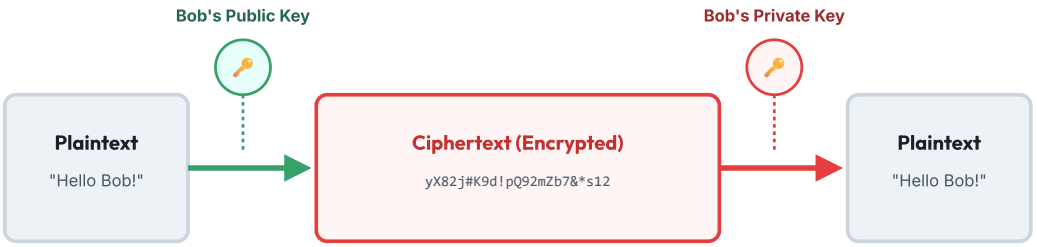

Figure 5.3: Asymmetric key encryption and decryption workflow.

Blockchain: A distributed, immutable ledger that records transactions across peer nodes. Blocks of data are linked using cryptographic hashes. Once linked, blocks cannot be modified without changing all subsequent blocks.

5.4 Internet of Things, Web Architecture, and Careers

Internet of Things (IoT): A network of physical objects embedded with sensors and actuators that collect and exchange environmental data over the internet.

Web and Mobile Development: Web development is split into Frontend (programming browser screens) and Backend (creating server APIs and database logic).

Career Opportunities: Software Engineer (designs application code structures), Cybersecurity Analyst (monitors threats), Data Scientist (analyzes business data patterns), and DevOps Engineer (automates deployments).

Module 5 - Practice Set & Exercises

1. Differentiate between SQL and NoSQL databases in terms of schema flexibility and scaling models.
2. Explain the ACID properties of databases (Atomicity, Consistency, Isolation, and Durability).
3. Explain how Machine Learning differs from traditional programming models.
4. Define the three principles of the CIA Triad in Cybersecurity.
5. Describe how public and private keys are used in asymmetric encryption to secure communications.
6. Explain how cryptographic hashes ensure data immutability in a blockchain architecture.
7. Describe an IoT architecture, explaining the roles of sensors, gateways, and actuators.
8. Contrast the career roles of a Software Engineer, a Data Scientist, and a DevOps Engineer.